CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA “PAULA SOUZA”

FACULDADE DE TECNOLOGIA DE RIBEIRÃO PRETO

CURSO SUPERIOR DE TECNOLOGIA EM

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RELATÓRIO TÉCNICO DO PROJETO DE BIBLIOTECA DE ESTRUTURA DE DADOS

LEANDRO DE SOUZA SERTORIO

Docente responsável: Profª Drª Heber Gustavo Xavier de Castro

RIBEIRÃO PRETO-SP

2026

SUMÁRIO

1. INTRODUÇÃO: 3
1.1 Objetivos do Projeto 3
1.2 Descrição Geral do Sistema 3
2. MODELAGEM 4
2.1 Estrutura dos dados 4
2.2 Organização das estruturas de dados 4
2.3 Fluxo de funcionamento do sistema 5
3. IMPLEMENTAÇÃO
3.1 Tecnologias utilizadas 5
3.2 Lista encadeada (funcionalidade extra) 5
3.3 Pilha 6
3.4 Fila 7
3.5 Árvore binária de busca 7
3.6 Algoritmo de ordenação (funcionalidade extra) 8
4. TESTES E RESULTADOS 8
4.1 Metodologia de testes 8
4.2 Casos de teste 9
4.3 Resultados obtidos 10
5. ANÁLISE DE COMPLEXIDADE 10
5.1 Complexidade das operações da lista encadeada 10
5.2 Complexidade das operações da pilha e fila 11
5.3 Complexidade das operações da árvore 12
5.4 Complexidade do algoritmo de ordenação 13
6. CONSIDERAÇÕES FINAIS 14

1. INTRODUÇÃO:

1.1 Objetivos do Projeto

O principal objetivo é aplicar os conceitos teóricos de estruturas de dados na construção de uma aplicação funcional em linguagem C.

Entre os objetivos específicos estão:

- Implementar estruturas de dados dinâmicas utilizando alocação de memória.
- Desenvolver operações de inserção, consulta e remoção de elementos.
- Compreender o funcionamento de listas encadeadas, pilhas, filas e árvores binárias de busca.
- Aplicar algoritmos de ordenação para organizar os registros.
- Integrar diferentes estruturas de dados em um mesmo programa.
- Desenvolver práticas de modularização, compilação e versionamento de código.

1.2 Descrição Geral do Sistema

O Sistema de Biblioteca permite cadastrar livros contendo identificador (ID), título, autor e ano de publicação.

Os registros são armazenados e manipulados por meio de diferentes estruturas de dados, cada uma com uma finalidade específica. O programa possui um menu interativo que permite ao usuário acessar as funcionalidades disponíveis.

Além das operações básicas de gerenciamento dos livros, o sistema disponibiliza uma fila de atendimento e um histórico das operações realizadas.

2. MODELAGEM

2.1 Estrutura dos dados

A principal entidade utilizada no sistema é o livro, representado por uma estrutura (struct) da linguagem C.

Cada livro possui quatro atributos:

ATRIBUTO	TIPO	DESCRIÇÃO
ID	int	Identificador Único do livro
Autor	char[80]	Autor da obra
Título	char[80]	Nome da obra
Ano	int	Ano de Publicação

A utilização de uma estrutura (struct) permite agrupar os atributos relacionados a cada livro em um único tipo de dado, facilitando seu armazenamento e manipulação pelas diferentes estruturas implementadas.

O identificador é utilizado também como referência para operações de busca, remoção e organização na árvore binária de busca.

2.2 Organização das estruturas de dados

O sistema utiliza diferentes estruturas para atender às necessidades de gerenciamento dos registros.

A lista encadeada é utilizada como estrutura principal para armazenar os livros cadastrados. A árvore binária de busca mantém os mesmos registros organizados pelo ID, permitindo operações de busca e consulta ordenadas.

A pilha é responsável por registrar as operações realizadas pelo usuário, enquanto a fila organiza os livros que aguardam atendimento.

O algoritmo Insertion Sort é utilizado para ordenar os livros pelo título. Para essa operação, os registros são copiados temporariamente para um vetor, preservando a organização original da lista.

2.3 Fluxo de funcionamento do sistema

O funcionamento começa pela inicialização das estruturas de dados. Em seguida, o programa apresenta um menu que permite ao usuário selecionar as operações desejadas.

Durante o cadastro, o sistema verifica se o identificador já existe. Caso não exista, o livro é inserido na lista e na árvore binária de busca.

As operações de remoção também atualizam as duas estruturas, evitando que permaneçam registros inconsistentes entre elas.

Quando um livro é adicionado à fila de atendimento, seus dados são copiados para a estrutura da fila. Dessa forma, a fila mantém sua própria cópia, independentemente de alterações posteriores no cadastro. Ao encerrar o programa, as estruturas dinâmicas são liberadas para evitar o desperdício de memória.

3. IMPLEMENTAÇÃO

3.1 Tecnologias utilizadas

O projeto foi desenvolvido utilizando a linguagem C, com o compilador GCC para realizar a compilação do código-fonte.

O desenvolvimento e a organização dos arquivos foram realizados no GitHub Codespaces, enquanto o Git e o GitHub foram utilizados para controle de versão e armazenamento remoto do projeto.

Também foi utilizado um Makefile para automatizar o processo de compilação, permitindo executar comandos simplificados para compilar, executar e limpar os arquivos gerados.

O projeto foi dividido em arquivos de cabeçalho (.h) e arquivos de implementação (.c), separando as declarações das funções de suas respectivas implementações.

3.2 Lista encadeada (funcionalidade extra)

A lista simplesmente encadeada é utilizada para armazenar os livros cadastrados no sistema.

Sua implementação utiliza nós dinâmicos, cada um contendo uma estrutura Livro e um ponteiro para o próximo elemento.

A lista possui dois ponteiros principais: inicio, que referencia o primeiro nó, e fim, que referencia o último nó.

Essa organização permite inserir novos livros no final da lista sem precisar percorrer todos os elementos para encontrar a última posição.

- Foram implementadas as seguintes operações:
- Inicialização da lista.
- Inserção de livros no final.
- Remoção de livros por ID.
- Exibição dos livros cadastrados.
- Liberação da memória utilizada.

A utilização da lista permite armazenar uma quantidade variável de livros, conforme a disponibilidade de memória, sem a necessidade de definir previamente um limite fixo de elementos.

3.3 Pilha

A pilha foi implementada utilizando uma lista simplesmente encadeada, seguindo o princípio LIFO (Last In, First Out), no qual o último elemento inserido é o primeiro a ser removido.

No sistema, sua finalidade é armazenar um histórico das operações realizadas pelo usuário, como cadastro, remoção, limpeza do cadastro e atendimento de livros.

Cada nó armazena uma descrição textual da operação e um ponteiro para o próximo elemento.

- Foram implementadas as operações:
- Inicialização da pilha.
- Inserção de uma ação (push).
- Remoção da ação mais recente (pop).
- Consulta da ação no topo (peek).
- Exibição das últimas cinco ações.
- Liberação da memória utilizada.

A pilha funciona como um histórico de operações. A remoção de um registro do histórico não desfaz a operação correspondente no sistema.

3.4 Fila

A fila foi implementada utilizando uma lista simplesmente encadeada, seguindo o princípio FIFO (First In, First Out), no qual o primeiro elemento inserido é o primeiro a ser removido.

No sistema, a fila representa o atendimento dos livros. O usuário pode selecionar um livro já cadastrado e adicioná-lo à fila para posterior atendimento.

Sua estrutura possui dois ponteiros: início, que referencia o primeiro elemento, e fim, que referencia o último.

- Foram implementadas as operações:
- Inicialização da fila.
- Inserção de livros no final (enfileirar).
- Remoção do primeiro elemento (desenfileirar).
- Exibição dos livros aguardando atendimento.
- Liberação da memória utilizada.

Um aspecto importante dessa implementação é que a fila armazena uma cópia dos dados do livro, em vez de um ponteiro para o registro original.

Assim, caso um livro seja removido da lista e da árvore enquanto ainda estiver aguardando atendimento, sua cópia permanece na fila até que seja atendida ou que a fila seja limpa.

3.5 Árvore binária de busca

A árvore binária de busca, também conhecida como BST (Binary Search Tree), foi implementada como uma funcionalidade adicional do projeto.

Sua organização utiliza o ID dos livros como chave de comparação. Cada nó possui os dados de um livro e dois ponteiros, um para o filho esquerdo e outro para o filho direito.

A regra de organização estabelece que os valores menores que a chave de um nó ficam à esquerda, enquanto os valores maiores ficam à direita.

Foram implementadas as seguintes operações:

- Inicialização da árvore.
- Inserção de livros.
- Busca de livros pelo ID.
- Remoção de nós.

- Percurso em ordem (in-order traversal).
- Consulta do menor ID.
- Consulta do maior ID.
- Liberação da memória utilizada.

A remoção considera três situações: nó sem filhos, nó com apenas um filho e nó com dois filhos. No último caso, é utilizado o sucessor em ordem, correspondente ao menor elemento da subárvore direita, para substituir o nó removido.

O percurso em ordem permite apresentar os livros em ordem crescente de ID.

3.6 Algoritmo de ordenação (funcionalidade extra)

Foi implementado o algoritmo Insertion Sort, também conhecido como ordenação por inserção, como funcionalidade adicional. O algoritmo organiza os livros em ordem alfabética crescente, utilizando o título como critério principal de comparação.

Para realizar a ordenação, os livros armazenados na lista são copiados temporariamente para um vetor. O algoritmo trabalha sobre esse vetor, preservando a estrutura original da lista. Durante a execução, cada elemento é comparado com os anteriores e deslocado até encontrar sua posição correta.

Em caso de títulos iguais, o ID é utilizado como critério secundário de ordenação, garantindo uma organização consistente.

Após a ordenação, os registros são exibidos ao usuário e a memória temporária do vetor é liberada.

4. TESTES E RESULTADOS

4.1 Metodologia de testes

Os testes foram definidos com o objetivo de verificar o funcionamento das operações implementadas e a integração entre as estruturas de dados.

Foram considerados cenários de utilização normal, situações de estruturas vazias, tentativas de operações inválidas e situações que envolvem a interação entre diferentes estruturas.

4.2 Casos de teste

ID	Procedimento	Resultado esperado
T01	Cadastrar um livro com dados válidos.	Livro inserido na lista e na árvore.
T02	Cadastrar dois livros com o mesmo ID.	Segundo cadastro recusado.
T03	Exibir a lista sem livros cadastrados.	Mensagem indicando lista vazia.
T04	Remover um livro existente.	Livro removido da lista e da árvore.
T05	Buscar um ID inexistente.	Mensagem de livro não encontrado.
T06	Consultar menor e maior ID.	Retorno dos livros correspondentes.
T07	Listar livros pela árvore.	Livros apresentados em ordem crescente de ID.
T08	Ordenar livros por título.	Livros apresentados em ordem alfabética.
T09	Inserir várias ações na pilha.	Histórico apresentado do mais recente para o mais antigo.
T10	Remover uma ação da pilha.	Ação mais recente retirada do histórico.
T11	Inserir vários livros na fila.	Livros mantidos na ordem de entrada.
T12	Atender o próximo livro.	Primeiro livro da fila removido.
T13	Atender com a fila vazia.	Mensagem indicando fila vazia.
T14	Remover do cadastro um livro que está na fila.	Livro removido da lista e árvore, mas mantido na fila.
T15	Atender o livro removido do cadastro.	Cópia do livro apresentado e removida da fila.
T16	Limpar o cadastro com livros na fila.	Lista, árvore e fila ficam vazias.

4.3 Resultados obtidos

ID	Resultado obtido	Status
T01	Sucesso, Aparece na árvore como na lista	Aprovado
T02	A mensagem de erro foi exibida	Aprovado
T03	A mensagem de lista vazia foi exibida	Aprovado
T04	A mensagem de lista e árvore vazia foram exibidas	Aprovado
T05	A mensagem de ID inexistente foi exibida	Aprovado
T06	Sucesso: aparecem como solicitados os livros de maior e menor ID.	Aprovado
T07	Os livros aparecem listados na árvore ordenados pela ID	Aprovado
T08	Os livros aparecem ordenados por título	Aprovado
T09	Ações aparecem no histórico	Aprovado
T10	Ação é removida, e sua ação foi constada fora do histórico	Aprovado
T11	Os livros estão ordenados por ordem de entrada	Aprovado
T12	Sucesso o próximo livro foi removido	Aprovado
T13	A mensagem de fila vazia foi exibida	Aprovado
T14	Sucesso, livro removido da lista e arvore mantido na fila.	Aprovado
T15	Sucesso, foi exibido uma mensagem de livro atendido e remoção na fila foi constada	Aprovado
T16	Foi exibido as mensagens indicando ausência de elementos	Aprovado

5. ANÁLISE DE COMPLEXIDADE

5.1 Complexidade das operações da lista encadeada

Considerando uma lista com n elementos e a implementação com ponteiros para o início e fim:

Operação	Complexidade
Inicialização	O(1)
Inserção no final	O(1)
Busca por ID	O(n)
Remoção por ID	O(n)
Exibição	O(n)
Liberar Memória	O(n)

A inserção no final possui complexidade constante porque a lista mantém um ponteiro direto para seu último nó. Já a busca e a remoção exigem percorrer os elementos até localizar o ID desejado.

5.2 Complexidade das operações da pilha e fila

A pilha utiliza uma estrutura encadeada e mantém um ponteiro para o elemento do topo.

Operação	Complexidade
Inicialização	O(1)
Inserção (push)	O(1)
Remoção (pop)	O(1)
Consulta do topo (peek)	O(1)
Exibição das últimas cinco ações	O(1)
Liberação da memória	O(n)

As operações de inserção, remoção e consulta do topo possuem complexidade constante, pois não precisam percorrer a estrutura.

A exibição das últimas cinco ações também possui custo limitado e independente da quantidade total de elementos armazenados.

A fila utiliza ponteiros para o primeiro e o último elemento, permitindo inserir e remover livros sem percorrer toda a estrutura.

Operação	Complexidade
Inicialização	O(1)
Inserção (enfileirar)	O(1)
Remoção (desenfileirar)	O(1)
Exibição dos elementos	O(n)
Liberação da memória	O(n)

A inserção e a remoção possuem complexidade constante devido à utilização dos ponteiros início e fim. A exibição e a liberação da fila exigem percorrer todos os elementos armazenados.

5.3 Complexidade das operações da árvore

A árvore binária de busca organiza os elementos de acordo com seus identificadores. Sua complexidade depende da altura da árvore, que pode variar conforme a ordem de inserção dos elementos.

Operação	Caso Médio	Pior Caso
Inserção	O(log n)	O(n)
Busca	O(log n)	O(n)
Remoção	O(log n)	O(n)
Percurso em Ordem	O(n)	O(n)
Consulta menor ID	O(log n)	O(n)
Consulta maior ID	O(log n)	O(n)

Em condições de distribuição equilibrada, a árvore apresenta operações de busca, inserção e remoção com complexidade média logarítmica. Entretanto, como a implementação não realiza balanceamento automático, a árvore pode assumir uma estrutura semelhante a uma lista encadeada caso os elementos sejam inseridos em ordem crescente ou decrescente. Nesse cenário, as operações podem atingir complexidade linear.

5.4 Complexidade do algoritmo de ordenação

Método: Insertion Sort

Cenário	Complexidade
Melhor caso	O(n)
Caso Médio	$O(n^2)$
Pior caso	$O(n^2)$

No melhor caso, quando os elementos já estão ordenados, o algoritmo realiza principalmente comparações.

Nos casos médio e pior, são necessários deslocamentos sucessivos dos elementos para encontrar suas posições corretas.

O algoritmo utiliza espaço auxiliar constante, O(1), para a ordenação propriamente dita. Entretanto, no sistema, também existe o vetor temporário utilizado para copiar os livros da lista, que ocupa O(n) de memória.

6. CONSIDERAÇÕES FINAIS

O desenvolvimento do Sistema de Biblioteca possibilitou aplicar os conceitos de estruturas de dados estudados na disciplina, demonstrando como diferentes formas de organização podem ser utilizadas para resolver problemas de armazenamento e manipulação de informações.

A implementação da lista encadeada, pilha e fila permitiu compreender o funcionamento de estruturas dinâmicas e suas respectivas operações. Além disso, a utilização da árvore binária de busca possibilitou explorar mecanismos de organização e pesquisa de dados por meio de uma estrutura hierárquica.

A implementação do Insertion Sort também permitiu aplicar conceitos de ordenação, analisando seu comportamento e sua complexidade computacional.

Outro aspecto importante foi a integração das estruturas em uma única aplicação, demonstrando que elas podem coexistir e desempenhar funções distintas dentro de um mesmo sistema.

Conclui-se que o projeto contribuiu para o desenvolvimento dos conhecimentos em programação C, alocação dinâmica de memória, modularização, manipulação de ponteiros e análise de algoritmos, além de proporcionar experiência prática na organização e documentação de um projeto de software.